

Técnicas para Explorar ILP

- **Técnicas estáticas:**

- Em geral, dependem do compilador para localizar ILP estaticamente, durante a **compilação** do programa

- **Técnicas dinâmicas:**

- Dependem do hardware para localizar e explorar ILP dinamicamente, durante a **execução** do programa

- **Técnicas:**

- Escalonamento dinâmico
- Predição dinâmica de desvios
- Despacho múltiplo dinâmico
- Especulação por hardware

Escalonamento Dinâmico

- **Limitação do pipeline simples:**

- **In-order issue, in-order execution** $\Rightarrow$ **Escalonamento estático**
- Se instrução sofre stall, nenhuma instrução seguinte pode prosseguir $\Rightarrow$ Perda de desempenho e unidades funcionais ociosas

- **Exemplo 1:**

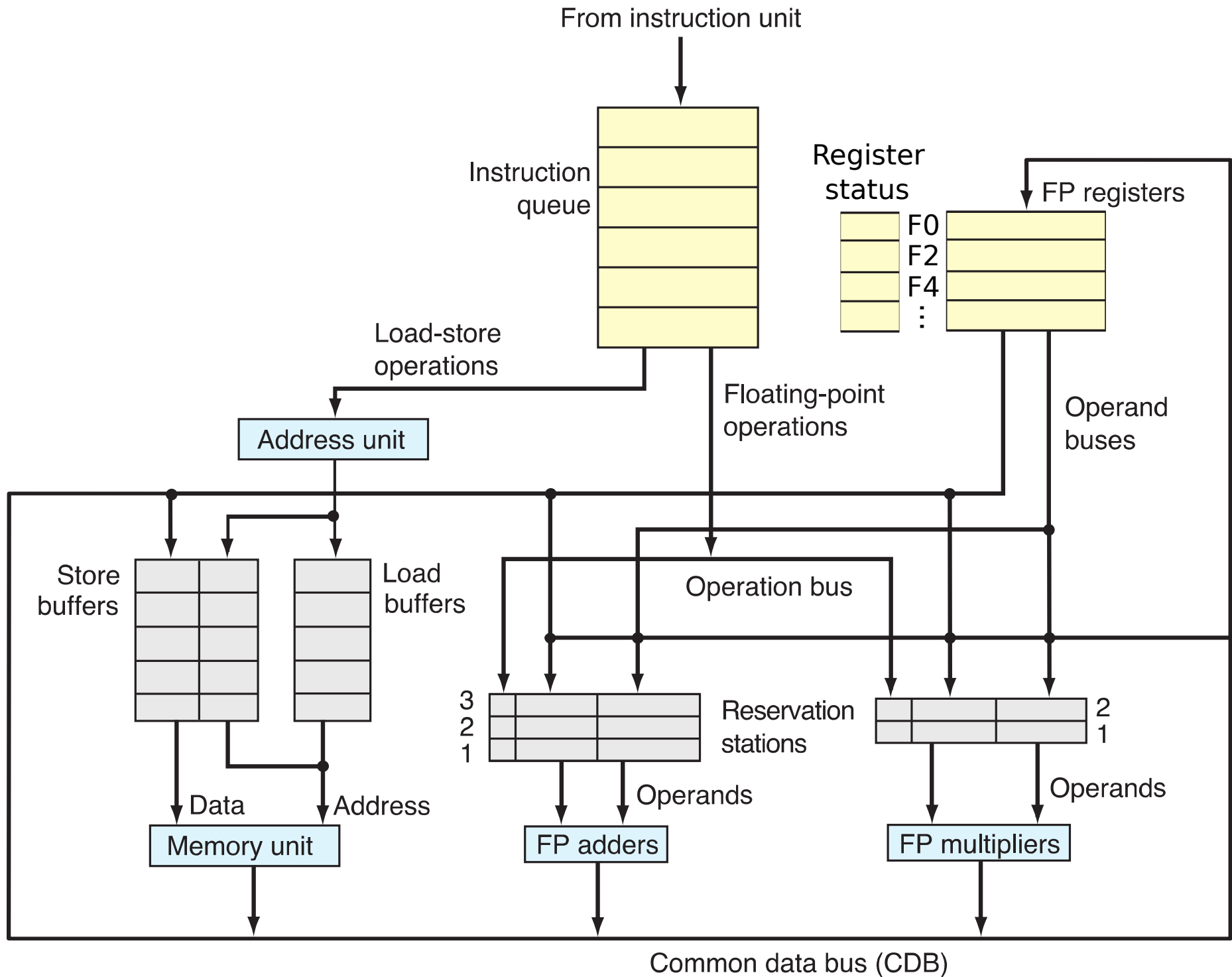
```
MUL.D  F0, F2, F4      # F0 = F2 * F4
ADD.D  F6, F0, F8      # F6 = F0 + F8
SUB.D  F0, F12, F14    # F0 = F12 - F14
```

- Supondo que multiplicação gasta vários ciclos em EX
- Dependência verdadeira $\text{MUL.D} \rightarrow \text{ADD.D (F0)}$:
 - **Conflito RAW** $\Rightarrow$ **Stall**
- Anti-dependência ADD.D/SUB.D (F0) :
 - Se executa SUB.D antes de $\text{ADD.D} \Rightarrow$ **Conflito WAR**
- Dependência de saída MUL.D/SUB.D (F0) :
 - Se executa SUB.D antes de $\text{MUL.D} \Rightarrow$ **Conflito WAW**

Escalonamento Dinâmico

- **Técnica em hardware:**
 - Durante execução do programa, processador muda ordem de execução das instruções, respeitando dependências verdadeiras
- **Várias instruções em execução ao mesmo tempo** (no estágio EX):
 - Múltiplas unidades funcionais (FUs) e/ou FUs “pipelined”
- **Objetivo:** Melhorar o desempenho
 - Reduz atrasos causados por dependências verdadeiras
 - Elimina atrasos causados por dependências falsas
- **In-order issue:** Detecta dependências entre as instruções
- **Out-of-order execution:** Instrução executa quando operandos estão disponíveis
- **Dificuldade:** Out-of-order execution $\Rightarrow$ **Out-of-order completion** $\Rightarrow$
Possibilidade de conflitos WAR e WAW
- **Técnicas de escalonamento dinâmico:**
 - **Scoreboard:** origem no processador do CDC 6600
 - **Algoritmo de Tomasulo:** origem no processador do IBM System 360/91
Processadores superscalares atuais usam variações do algoritmo de Tomasulo

Exemplo: Processador MIPS com Algoritmo de Tomasulo



Exemplo: Processador MIPS com Algoritmo de Tomasulo

- **Despacho simples** e **in-order**
- **Fila de instruções**: ordem FIFO
- Conjunto de registradores de ponto flutuante
- **Unidades funcionais** (FUs):
 - FP adders: operações soma e subtração
 - FP multipliers: operações multiplicação e divisão
 - Unidade de memória: operações load e store
- **Reservation stations** (RSs) das FUs:
 - Armazenam instruções em execução e em stall, aguardando operandos ou FU
 - Controlam execução das instruções, detectam e resolvem conflitos
 - Mantêm estrutura de dados interna com estado da instrução
- **Load buffers** e **store buffers**: RSs da unidade de memória
- **Barramento Common Data Bus** (CDB):
 - Resultados das FUs e da unidade de memória (`load`) vão para CDB
 - Do CDB, resultado é encaminhado para RSs, store buffers e conjunto de registradores
 - Resultado pode ser encaminhado para várias unidades ao mesmo tempo

Estruturas de Dados do Algoritmo de Tomasulo

- Usadas para controlar execução das instruções
- Estruturas e campos:
 - **Status dos Reservation Stations e dos load/store buffers (RS):**
 - **Busy** : RS está ocupada
 - **Op** : operação a realizar
 - **D** : registrador destino
 - **Vj, Vk** : valor dos operandos fonte1 e fonte2
 - **Qj, Qk** : RS que produzirá operandos fonte1 e fonte2
 - **A** (apenas para load/store buffers) : valor imediato ou endereço
 - **Status dos registradores:**
 - **Qi** : RS que produzirá resultado a ser escrito no registrador

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
0	N							
1	N							
⋮								

Register Status

Reg	Qi
0	—
1	—
⋮	

Unidade Integrada de Busca de Instruções

- Busca instruções na memória e insere na fila de instruções, na ordem FIFO

Controle do Algoritmo de Tomasulo

- **Passos de execução da instrução:**
 - **Estágio II:** Instruction Issue (despacho da instrução)
 - **Estágio EX:** Execution
 - **Estágio MEM:** Memory access (apenas load e store)
 - **Estágio WR:** Write Result (exceto store)

Estágio II

- **Stall enquanto:**
 - Não há RS livre para 1ª instrução da fila de instruções: **conflito estrutural**
- **Ações:**
 - Retira 1ª instrução da fila e despacha para RS **r** (ou load/store buffer): **in-order issue**
 - Para cada reg fonte da instrução:
 - Se operando está atual no conjunto de registradores:
 - Copia seu valor para RS **r**: **armazenamento temporário**
 - Senão: **conflito RAW**
 - Anota na RS **r** qual RS produzirá valor: **renomeação de registradores**
 - Para reg destino da instrução:
 - Anota em Register Status que RS **r** produzirá valor desse operando

Estágio II

- Ações:

Operação FP

opfp dst, fnt1, fnt2

Busy [r] := Y

Op [r] := ...

D [r] := dst

Se Qi [fnt1] $\neq$ — Então

Qj [r] := Qi [fnt1]

Senão

Vj [r] := Reg [fnt1]

Qj [r] := —

Se Qi [fnt2] $\neq$ — Então

Qk [r] := Qi [fnt2]

Senão

Vk [r] := Reg [fnt2]

Qk [r] := —

Qi [dst] := r

Load

load dst, imm (fnt1)

Busy [r] := Y

D [r] := dst

Se Qi [fnt1] $\neq$ — Então

Qj [r] := Qi [fnt1]

Senão

Vj [r] := Reg [fnt1]

Qj [r] := —

A [r] := imm

Qi [dst] := r

Store

store fnt2, imm (fnt1)

Busy [r] := Y

Se Qi [fnt1] $\neq$ — Então

Qj [r] := Qi [fnt1]

Senão

Vj [r] := Reg [fnt1]

Qj [r] := —

Se Qi [fnt2] $\neq$ — Então

Qk [r] := Qi [fnt2]

Senão

Vk [r] := Reg [fnt2]

Qk [r] := —

A [r] := imm

Estágio EX: Operação FP

- **Stall enquanto:**

- Operando **fonte1** ou **fonte2** da instrução não está disponível na RS:

$$Q_j[r] \neq \text{—} \quad \text{or} \quad Q_k[r] \neq \text{—}$$

conflito RAW

- Monitora CDB, até valor do operando ser encaminhado

- **Ações:**

- Copia valor do operando do CDB para $V_j[r]$ ou $V_k[r]$ na RS:

armazenamento temporário

- **Stall enquanto:**

- Não há FU livre para instrução: **conflito estrutural**

- **Ações:**

- Executa operação na FU com operandos $V_j[r]$ e $V_k[r]$
(pode levar vários ciclos)

Estágio EX: Load / Store

- **Stall enquanto:**

- Operando **fonte1** da instrução não está disponível na RS: **conflito RAW**

$$Q_j[r] \neq \text{—}$$

- Monitora CDB, até valor do operando ser encaminhado

- **Ações:**

- Copia valor do operando do CDB para $V_j[r]$ na RS:

armazenamento temporário

- Calcula endereço de acesso à memória e guarda no load/store buffer

$$A[r] := V_j[r] + A[r]$$

Estágio MEM: Load / Store

- **Stall enquanto:**
 - **Store:**
 - Operando **fonte2** da instrução não está disponível na RS: **conflito RAW**
 $Q_k[r] \neq \text{—}$
 - Monitora CDB, até valor do operando ser encaminhado
- **Ações:**
 - **Store:**
 - Copia valor do operando do CDB para $V_k[r]$ na RS: **armazen. temporário**
- **Stall enquanto:**
 - Unidade de acesso à memória está ocupada: **conflito estrutural**
 - Conflito de dados pela memória
- **Ações:**
 - **Load:**
 - Realiza leitura do endereço $A[r]$ da memória
 - **Store:**
 - Realiza escrita do valor $V_k[r]$ no endereço $A[r]$ da memória
 - $\text{Busy}[r] := N$

Estágio WR: Operação FP e Load

- **Stall enquanto:**
 - CDB não está disponível: **conflito estrutural**
- **Ações:**
 - Escreve resultado no CDB
 - Do CDB, resultado é encaminhado para:
 - RSs aguardando operando: **forwarding** (e store buffers)
 - Conjunto de registradores:
 - Se Register Status indicar RS r , então atualiza

Operação FP e Load:

Escreve resultado no CDB

Se $Qi[D[r]] = r$ Então

$Reg[D[r]] := resultado$

$Qi[D[r]] := \text{—}$

Para todo RS s :

Se $Qj[s] = r$ Então

$Vj[s] := resultado$

$Qj[s] := \text{—}$

Se $Qk[s] = r$ Então

$Vk[s] := resultado$

$Qk[s] := \text{—}$

$Busy[r] := N$

Exemplos: Algoritmo de Tomasulo

- **Estágios:**

- **II**: 1 ciclo
- **EX**:
 - Load/Store: 1 ciclo (cálculo de endereço)
 - Soma/Subtração FP: 2 ciclos
 - Multiplicação/Divisão FP: 10 ciclos
- **MEM**: 1 ciclo (apenas Load/Store)
- **WR**: 1 ciclo (exceto Store)

- **1 CDB:**

- Resultado no CDB pode ser usado por outra instrução no mesmo ciclo

- **RSs:**

Add	Mult	Load	Store
2	2	2	2

- **FUs:**

Somador/Subtrador	Multiplicador/Divisor	Unidade de acesso à Memória
2	2	1

Exemplo 2: Execução – Ciclo 1

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
L.D	F2,	16	(R2)	1				Despacha
MUL.D	F0,	F2,	F4					
DIV.D	F10,	F0,	F6					
ADD.D	F8,	F8,	F4					
S.D	F8,	24	(R2)					

Dependências verdadeiras:

- L.D → MUL.D (F2)
- MUL.D → DIV.D (F0)
- ADD.D → S.D (F8)

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	Y	load	F2	40	—	—	—	16
Store1	N	—	—	—	—	—	—	—
Add1	N	—	—	—	—	—	—	—
Mult1	N	—	—	—	—	—	—	—
Mult2	N	—	—	—	—	—	—	—

R2 = 40 Mem[56] = 50.0
 Mem[64] =

Register Status

Reg	Qi
F0	—
F2	Load1
F4	—
F6	—
F8	—
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	
F4	20.0
F6	40.0
F8	10.0
F10	
...	

Exemplo 2: Execução – Ciclo 2

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
L.D F2, 16 (R2)	1	2			Executa (cálculo de endereço) Despacha
MUL.D F0, F2, F4	2				
DIV.D F10, F0, F6					
ADD.D F8, F8, F4					
S.D F8, 24 (R2)					

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	Y	load	F2	40	—	—	—	56
Store1	N	—	—	—	—	—	—	—
Add1	N	—	—	—	—	—	—	—
Mult1	Y	mul	F0	—	20.0	Load1	—	—
Mult2	N	—	—	—	—	—	—	—

R2 = 40

Mem[56] = 50.0

Mem[64] =

Register Status

Reg	Qi
F0	Mul1
F2	Load1
F4	—
F6	—
F8	—
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	
F4	
F6	
F8	
F10	10.0
...	...

Exemplo 2: Execução – Ciclo 3

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
L.D	F2,	16	(R2)	1	2	3		Lê da memória
MUL.D	F0,	F2,	F4	2	(3)			Em stall: conflito RAW com Load1
DIV.D	F10,	F0,	F6	3				Despacha
ADD.D	F8,	F8,	F4					
S.D	F8,	24	(R2)					

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	Y	load	F2	40	—	—	—	56
Store1	N	—	—	—	—	—	—	—
Add1	N	—	—	—	—	—	—	—
Mult1	Y	mul	F0	—	20.0	Load1	—	—
Mult2	Y	div	F10	—	40.0	Mult1	—	—

R2 = 40

Mem[56] = 50.0

Mem[64] =

Register Status

Reg	Qi
F0	Mult1
F2	Load1
F4	—
F6	—
F8	—
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	
F4	20.0
F6	40.0
F8	10.0
F10	
...	

Exemplo 2: Execução – Ciclo 4

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
L.D F2, 16 (R2)	1	2	3	4	Escreve resultado: forwarding p/ Mult1 Executa (passo 1) Em stall: conflito RAW com Mult1 Despacha
MUL.D F0, F2, F4	2	(3)-4			
DIV.D F10, F0, F6	3	(4)			
ADD.D F8, F8, F4	4				
S.D F8, 24 (R2)					

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	N	—	—	—	—	—	—	—
Store1	N	—	—	—	—	—	—	—
Add1	Y	add	F8	10.0	20.0	—	—	—
Mult1	Y	mul	F0	50.0	20.0	—	—	—
Mult2	Y	div	F10	—	40.0	Mult1	—	—

R2 = 40

Mem[56] = 50.0

Mem[64] =

Register Status

Reg	Qi
F0	Mult1
F2	—
F4	—
F6	—
F8	Add1
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	50.0
F4	20.0
F6	40.0
F8	10.0
F10	
...	

Exemplo 2: Execução – Ciclo 5

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
L.D	F2,	16	(R2)	1	2	3	4	—
MUL.D	F0,	F2,	F4	2	(3)-4-5			Executa (passo 2)
DIV.D	F10,	F0,	F6	3	(4-5)			Em stall: conflito RAW com Mult1
ADD.D	F8,	F8,	F4	4	5			Executa (passo 1)
S.D	F8,	24	(R2)	5				Despacha

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	N	—	—	—	—	—	—	—
Store1	Y	store	—	40	—	—	Add1	24
Add1	Y	add	F8	10.0	20.0	—	—	—
Mult1	Y	mul	F0	50.0	20.0	—	—	—
Mult2	Y	div	F10	—	40.0	Mult1	—	—

R2 = 40 Mem[56] = 50.0
 Mem[64] =

Register Status

Reg	Qi
F0	Mult1
F2	—
F4	—
F6	—
F8	Add1
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	50.0
F4	20.0
F6	40.0
F8	10.0
F10	
...	

Exemplo 2: Execução – Ciclo 6

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
L.D	F2,	16	(R2)	1	2	3	4	—
MUL.D	F0,	F2,	F4	2	(3)-4-6			Executa (passo 3)
DIV.D	F10,	F0,	F6	3	(4-6)			Em stall: conflito RAW com Mult1
ADD.D	F8,	F8,	F4	4	5-6			Executa (passo 2)
S.D	F8,	24	(R2)	5	6			Executa (cálculo de endereço)

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	N	—	—	—	—	—	—	—
Store1	Y	store	—	40	—	—	Add1	64
Add1	Y	add	F8	10.0	20.0	—	—	—
Mult1	Y	mul	F0	50.0	20.0	—	—	—
Mult2	Y	div	F10	—	40.0	Mult1	—	—

R2 = 40 Mem[56] = 50.0
 Mem[64] =

Register Status

Reg	Qi
F0	Mult1
F2	—
F4	—
F6	—
F8	Add1
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	50.0
F4	20.0
F6	40.0
F8	10.0
F10	
...	

Exemplo 2: Execução – Ciclo 7

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
L.D	F2,	16	(R2)	1	2	3	4	—
MUL.D	F0,	F2,	F4	2	(3)-4-7			Executa (passo 4)
DIV.D	F10,	F0,	F6	3	(4-7)			Em stall: conflito RAW com Mult1
ADD.D	F8,	F8,	F4	4	5-6	—	7	Escreve resultado: forwarding p/ Store1
S.D	F8,	24	(R2)	5	6	7	—	Escreve na memória

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	N	—	—	—	—	—	—	—
Store1	Y	store	—	40	30.0	—	—	64
Add1	N	—	—	—	—	—	—	—
Mult1	Y	mul	F0	50.0	20.0	—	—	—
Mult2	Y	div	F10	—	40.0	Mult1	—	—

R2 = 40

Mem[56] = 50.0

Mem[64] =

Register Status

Reg	Qi
F0	Mult1
F2	—
F4	—
F6	—
F8	—
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	50.0
F4	20.0
F6	40.0
F8	30.0
F10	
...	

Exemplo 2: Execução – Ciclo 7 (continuação)

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
L.D	F2,	16	(R2)	1	2	3	4	—
MUL.D	F0,	F2,	F4	2	(3)-4-7			Executa (passo 4)
DIV.D	F10,	F0,	F6	3	(4-7)			Em stall: conflito RAW com Mult1
ADD.D	F8,	F8,	F4	4	5-6	—	7	Escreve resultado: forwarding p/ Store1
S.D	F8,	24	(R2)	5	6	7	—	Escreve na memória

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	N	—	—	—	—	—	—	—
Store1	N	—	—	—	—	—	—	—
Add1	N	—	—	—	—	—	—	—
Mult1	Y	mul	F0	50.0	20.0	—	—	—
Mult2	Y	div	F10	—	40.0	Mult1	—	—

R2 = 40

Mem[56] = 50.0

Mem[64] = 30.0

Register Status

Reg	Qi
F0	Mult1
F2	—
F4	—
F6	—
F8	—
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	50.0
F4	20.0
F6	40.0
F8	30.0
F10	
...	

Exemplo 2: Execução – Ciclos 8 a 13

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
L.D F2, 16 (R2)	1	2	3	4	—
MUL.D F0, F2, F4	2	(3)-4-8-13			Executa (passos 5-10) Em stall: conflito RAW com Mult1
DIV.D F10, F0, F6	3	(4-8-13)			
ADD.D F8, F8, F4	4	5-6	—	7	—
S.D F8, 24 (R2)	5	6	7	—	—

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	N	—	—	—	—	—	—	—
Store1	N	—	—	—	—	—	—	—
Add1	N	—	—	—	—	—	—	—
Mult1	Y	mul	F0	50.0	20.0	—	—	—
Mult2	Y	div	F10	—	40.0	Mult1	—	—

R2 = 40

Mem[56] = 50.0

Mem[64] = 30.0

Register Status

Reg	Qi
F0	Mult1
F2	—
F4	—
F6	—
F8	—
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	
F2	50.0
F4	20.0
F6	40.0
F8	30.0
F10	
...	

Exemplo 2: Execução – Ciclo 14

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
L.D F2, 16 (R2)	1	2	3	4	—
MUL.D F0, F2, F4	2	(3)-4-13	—	14	Escreve resultado: forwarding p/ Mult2 Executa (passo 1)
DIV.D F10, F0, F6	3	(4-13)-14	—	7	
ADD.D F8, F8, F4	4	5-6	—	7	—
S.D F8, 24 (R2)	5	6	7	—	—

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Load1	N	—	—	—	—	—	—	—
Store1	N	—	—	—	—	—	—	—
Add1	N	—	—	—	—	—	—	—
Mult1	N	—	—	—	—	—	—	—
Mult2	Y	div	F10	1000.0	40.0	—	—	—

R2 = 40

Mem[56] = 50.0

Mem[64] = 30.0

Register Status

Reg	Qi
F0	—
F2	—
F4	—
F6	—
F8	—
F10	Mult2
...	—

Conjunto de Registradores

Reg	Valor
F0	1000.0
F2	50.0
F4	20.0
F6	40.0
F8	30.0
F10	
...	

Algoritmo de Tomasulo: Vantagens e Desvantagens

- **Vantagens:**
 - **Redução dos atrasos por conflitos RAW:**
 - Out-of-order execution
 - Forwarding pelo CDB para RSs
 - **Eliminação dos atrasos por conflitos WAR e WAW:**
 - Renomeação de registradores
 - Armazenamento dos operandos nas RSs
 - **Distribuição da detecção e controle de conflitos:**
 - Resultado pode ser encaminhado para várias RSs ao mesmo tempo
 - **Dynamic memory disambiguation:**
 - Trata dependências não conhecidas durante compilação (pela memória)
 - Compara endereços de memória de instruções `load/store` $\Rightarrow$
Detecta e trata conflitos RAW, WAR e WAW pela memória
 - **Execução sobreposta de iterações de um laço** (com especulação por hw)
 - **Compilador mais simples**
- **Desvantagem?**

Exemplo 3: Execução com Dependências Falsas – Ciclo 1

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1				Despacha
DIV.D F8, F10, F2					
SUB.D F2, F6, F4					
ADD.D F10, F6, F4					

- Dependência verdadeira:
 - MUL.D $\rightarrow$ DIV.D (F2)
- Anti-dependências:
 - DIV.D / SUB.D (F2)
 - DIV.D / ADD.D (F10)
- Dependência de saída:
 - MUL.D / SUB.D (F2)

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	N	—	—	—	—	—	—	—

Register Status

Reg	Qi
F2	Mult1
F4	—
F6	—
F8	—
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	
F4	2.0
F6	6.0
F8	
F10	36.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclo 2

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1	2			Executa (passo 1) Despacha
DIV.D F8, F10, F2	2				
SUB.D F2, F6, F4					
ADD.D F10, F6, F4					

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	Y	div	F8	36.0	—	—	Mult1	—

Register Status

Reg	Qi
F2	Mult1
F4	—
F6	—
F8	Mult2
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	
F4	2.0
F6	6.0
F8	
F10	36.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclo 3

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1	2-3			Executa (passo 2) Em stall: conflito RAW com Mult1 Despacha
DIV.D F8, F10, F2	2	(3)			
SUB.D F2, F6, F4	3				
ADD.D F10, F6, F4					

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	Y	sub	F2	6.0	2.0	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	Y	div	F8	36.0	—	—	Mult1	—

Register Status

Reg	Qi
F2	Add1
F4	—
F6	—
F8	Mult2
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	
F4	2.0
F6	6.0
F8	
F10	36.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclo 4

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1	2-4			Executa (passo 3)
DIV.D F8, F10, F2	2	(3-4)			Em stall: conflito RAW com Mult1
SUB.D F2, F6, F4	3	4			Executa (passo 1)
ADD.D F10, F6, F4	4				Despacha

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	Y	sub	F2	6.0	2.0	—	—	—
Add2	Y	add	F10	6.0	2.0	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	Y	div	F8	36.0	—	—	Mult1	—

Register Status

Reg	Qi
F2	Add1
F4	—
F6	—
F8	Mult2
F10	Add2
...	—

Conjunto de Registradores

Reg	Valor
F2	
F4	2.0
F6	6.0
F8	
F10	36.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclo 5

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
MUL.D	F2,	F4,	F6	1	2-5			Executa (passo 4)
DIV.D	F8,	F10,	F2	2	(3-5)			Em stall: conflito RAW com Mult1
SUB.D	F2,	F6,	F4	3	4-5			Executa (passo 2)
ADD.D	F10,	F6,	F4	4	5			Executa (passo 1)

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	Y	sub	F2	6.0	2.0	—	—	—
Add2	Y	add	F10	6.0	2.0	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	Y	div	F8	36.0	—	—	Mult1	—

Register Status

Reg	Qi
F2	Add1
F4	—
F6	—
F8	Mult2
F10	Add2
...	—

Conjunto de Registradores

Reg	Valor
F2	
F4	2.0
F6	6.0
F8	
F10	36.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclo 6

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1	2-6			Executa (passo 5)
DIV.D F8, F10, F2	2	(3-6)			Em stall: conflito RAW com Mult1
SUB.D F2, F6, F4	3	4-5	—	6	Escreve result.: conflitos WAW (Mult1), WAR (Mult2)
ADD.D F10, F6, F4	4	5-6			Executa (passo 2)

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	Y	add	F10	6.0	2.0	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	Y	div	F8	36.0	—	—	Mult1	—

Register Status

Reg	Qi
F2	—
F4	—
F6	—
F8	Mult2
F10	Add2
...	—

Conjunto de Registradores

Reg	Valor
F2	4.0
F4	2.0
F6	6.0
F8	
F10	36.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclo 7

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1	2-7			Executa (passo 6) Em stall: conflito RAW com Mult1
DIV.D F8, F10, F2	2	(3-7)			
SUB.D F2, F6, F4	3	4-5	—	6	—
ADD.D F10, F6, F4	4	5-6	—	7	Escreve resultado: conflito WAR (Mult2)

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	Y	div	F8	36.0	—	—	Mult1	—

Register Status

Reg	Qi
F2	—
F4	—
F6	—
F8	Mult2
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	4.0
F4	2.0
F6	6.0
F8	
F10	8.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclos 8 a 11

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
MUL.D	F2,	F4,	F6	1	2-8-11			Executa (passos 7-10) Em stall: conflito RAW com Mult1
DIV.D	F8,	F10,	F2	2	(3-8-11)			
SUB.D	F2,	F6,	F4	3	4-5	—	6	—
ADD.D	F10,	F6,	F4	4	5-6	—	7	—

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	Y	mul	F2	2.0	6.0	—	—	—
Mult2	Y	div	F8	36.0	—	—	Mult1	—

Register Status

Reg	Qi
F2	—
F4	—
F6	—
F8	Mult2
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	4.0
F4	2.0
F6	6.0
F8	
F10	8.0
...	

Exemplo 3: Execução com Dependências Falsas – Ciclo 12

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1	2-11	—	12	Escreve resultado: forwarding p/ Mult2 Executa (passo 1)
DIV.D F8, F10, F2	2	(3-11)-12	—	—	
SUB.D F2, F6, F4	3	4-5	—	6	—
ADD.D F10, F6, F4	4	5-6	—	7	—

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	N	—	—	—	—	—	—	—
Mult2	Y	div	F8	36.0	12.0	—	—	—

Register Status

Reg	Qi
F2	—
F4	—
F6	—
F8	Mult2
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	4.0
F4	2.0
F6	6.0
F8	—
F10	8.0
...	—

Exemplo 3: Execução com Dependências Falsas – Ciclos 13 a 21

Instrução				Estágios/Ciclos				Observação
				II	EX	Mem	WR	
MUL.D	F2,	F4,	F6	1	2-11	—	12	—
DIV.D	F8,	F10,	F2	2	(3-11)-12-13-21	—	—	Executa (passos 2-10)
SUB.D	F2,	F6,	F4	3	4-5	—	6	—
ADD.D	F10,	F6,	F4	4	5-6	—	7	—

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	N	—	—	—	—	—	—	—
Mult2	Y	div	F8	36.0	12.0	—	—	—

Register Status

Reg	Qi
F2	—
F4	—
F6	—
F8	Mult2
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	4.0
F4	2.0
F6	6.0
F8	—
F10	8.0
...	—

Exemplo 3: Execução com Dependências Falsas – Ciclo 22

Instrução	Estágios/Ciclos				Observação
	II	EX	Mem	WR	
MUL.D F2, F4, F6	1	2-11	—	12	—
DIV.D F8, F10, F2	2	(3-11)-12-21	—	22	Escreve resultado
SUB.D F2, F6, F4	3	4-5	—	6	—
ADD.D F10, F6, F4	4	5-6	—	7	—

Reservation Station Status

RS	Busy	Op	D	Vj	Vk	Qj	Qk	A
Add1	N	—	—	—	—	—	—	—
Add2	N	—	—	—	—	—	—	—
Mult1	N	—	—	—	—	—	—	—
Mult2	N	—	—	—	—	—	—	—

Register Status

Reg	Qi
F2	—
F4	—
F6	—
F8	—
F10	—
...	—

Conjunto de Registradores

Reg	Valor
F2	4.0
F4	2.0
F6	6.0
F8	3.0
F10	8.0
...	...